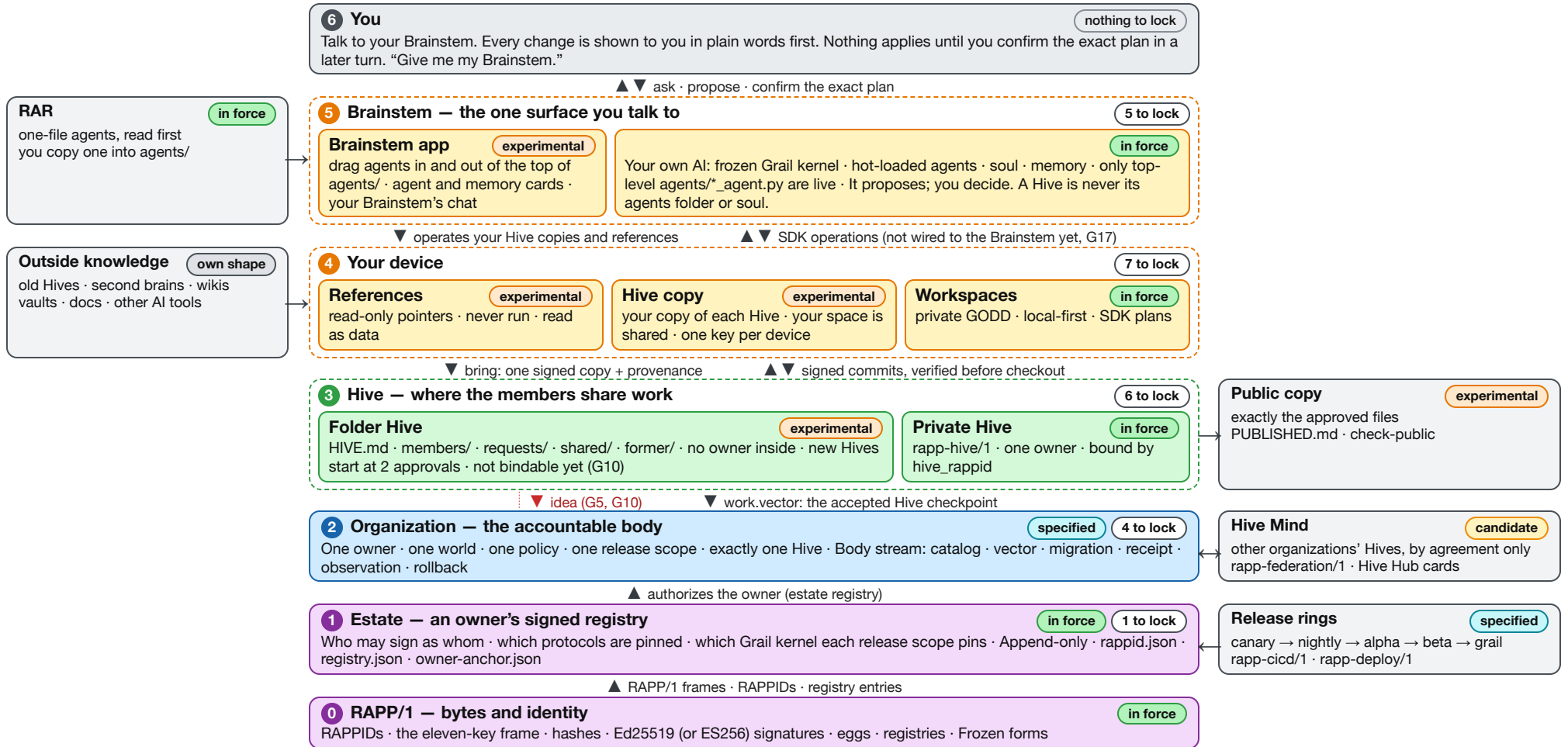


The RAPP/1 organism, on one page

Read it top (6) to bottom (0). Left: what comes in. Right: what goes out or across.



Dogfood: the RAPP Hive, our public example

HIVE.md	members: the maintainers (AIs are tools)
shared/organism/canon/	one canonical fact per file, hash-stamped
shared/organism/drift/	one verdict per repo: aligned or drifted
shared/organism/map/	layers 0-6 · crossings · this page
shared/organism/gaps/	one file per gap
(your device)	references: every estate repo, read-only

```

graph LR
    1["1 Canon  
one fact, once, from its repo"] --> 2["2 Sweep  
rapp_check.py, not in the Hive"]
    2 --> 3["3 Record  
one verdict per repo, signed"]
    3 --> 4["4 Fix upstream  
the next sweep flips it"]
  
```

1 Canon
one fact, once, from its repo

2 Sweep
rapp_check.py, not in the Hive

3 Record
one verdict per repo, signed

4 Fix upstream
the next sweep flips it

Built locally; public copy: `kody-w/rapp-hive-public`. Its first sweep ran `rapp-1`'s own `rapp_check.py` over 10 public estate repos: 7 were compliant or clean and 3 showed drift (non-conformant eggs in RAPP and RAR; frozen frames in `rapp-model-hive`'s main that cannot be signature-checked, gap G8). Each verdict is one signed file. **How it scales:** the same one agent and the same rules run the 6-person Contoso model and 10 real repos, on the way to about 30.

The path to a locked RAPP/1

1 **Decide** · you **G8** **G9** **G18** **G19** **G21**

2 **Zero drift** · engineering 3 steps, no gaps

3 **Close the specification gaps** · spec owner
G1 **G2** **G3** **G4** **G5** **G6** **G7** **G10** **G11** **G22**

4 **Switch it on** · estate owner **G15** **G16** **G17**

5 **Graduate and freeze** · engineering
G12 **G13** **G14** **G20**

RAPP/1 is locked when the whole stack is frozen, pinned, in force and free of drift, and nothing experimental is left undecided. Five phases get there, in order; the owner's decisions come first.

What holds everywhere

- RAPP/1 wins, then rapp-work/1.
- Transport carries; signatures decide.
- In by signed copy, out by approved copy.
- Other people's text is data.
- Propose, confirm, apply.
- Accepted history is never rewritten.
- The Brainstem is the one surface you talk to.
- Only top-level agents/*_agent.py files are live.

The path to a locked RAPP/1

What we need to do to lock in a full RAPP/1: five phases, in order, with the owner's decisions first.



Your decisions first · phase 1, Decide · you, about an hour

1. merge RAPP proposal 0001 (G18)

2. pick the kernel to lock (G21)

3. decide Tier 2 loader parity (G19)
4. ratify the RAPP Work Constitution, with its Part V proposals (G8, G9)

5. choose the experimental parts in the lock: Folder Hive and its public copy, References, Hub cards, Brainstem app

RAPP/1 is locked when

- every normative specification in the stack is frozen at a version and pinned in one signed estate registry
- each one is in force
- the estate sweep shows zero drift
- every gap is closed, or deferred to a later RAPP with the owner's sign-off
- the Constitutions are ratified
- every experimental part has graduated through the rings, or is marked as outside the lock

Lock status by layer, top to bottom

6 You	nothing to lock	
5 Brainstem	5 to lock	G17G18G19G20G21
4 Your device	7 to lock	G2G3G4G7G13G14G22
3 Hive	6 to lock	G1G6G8G9G12Public copy
2 Organization	4 to lock	G5G10G11G16
1 Estate	1 to lock	G15
0 RAPP/1	in force	

2 · Zero drift

- engineering
- fix RAPP's and RAR's non-conformant eggs, and RAR's bounded scan
 - give rapp-model-hive's frames a trusted anchor
 - re-sweep until all 10 repos are clean (the dogfood)

3 · Close the specification gaps

- spec owner
- rapp-hive/1 (G1, G6)
 - rapp-work-sdk/1 (G2, G3, G4, G7)
 - rapp-work/1 (G5, G10, G11, G22)
- G1G2G3G4G5G6G7G10G11G22

4 · Switch it on

- estate owner
- sign the grail-kernel entry (G15)
 - activate rapp-work/1 and its pins, release rings included (G16)
 - give the Brainstem an SDK agent (G17)
- G15G16G17

5 · Graduate and freeze

- engineering
- a real team's week for the Hive agent (G14) and the Brainstem app (G20), then the rings
 - graduate Hub cards (G12) and References (G13), or mark them outside the lock
 - merge the experimental branches
 - tag the lock across repositories
 - run the drift sweep on a schedule
- G12G13G14G20

Gap register: each gap, its phase and who acts

G1	A rapp-hive/1 roster can never change	open	3 · spec owner	G12	Hive Hub dial records did not describe folder Hives	experimental	5 · engineering
G2	SDK plans cannot move a file	open	3 · spec owner	G13	Outside knowledge had no way in	experimental	5 · engineering
G3	SDK discovery misses *_agent.py	open	3 · spec owner	G14	The Hive agent's path to the production ring	open	5 · engineering
G4	SDK migration refuses repository-seeded Hives and long world ids	open	3 · spec owner	G15	No estate declares the Brainstem's kernel	open	4 · estate owner
G5	One owner only, so co-equal groups do not fit	idea	3 · spec owner	G16	No estate has activated canonical rapp-work/1	open	4 · estate owner
G6	Implementations refuse owner succession	open	3 · spec owner	G17	The Brainstem does not call the RAPP Work SDK	idea	4 · estate owner
G7	SDK verification accepts an edited instruction file	open	3 · spec owner	G18	RAPP's Constitution said agents load from every folder	proposed	1 · you
G8	Hive members are names bound to keys, not RAPPIDs	proposed	1 · you	G19	The cloud Brainstem loads agents differently	open	1 · you
G9	A git commit is not authority in canonical rapp-work/1	proposed	1 · you	G20	The Brainstem app's path to the production ring	open	5 · engineering
G10	An organization cannot bind a folder Hive	idea	3 · spec owner	G21	RAPP's kernel pin lags the Grail kernel's newest release	open	1 · you
G11	"Organization" means two things	idea	3 · spec owner	G22	"Workspace" names two things	idea	3 · spec owner